

Architecture Checkpoint Report

Assignment 2: Architectural Evolution of nopCommerce

Group: 107

Members: Afonso Ferreira (113480), Tomás Brás (112665), Hugo Ribeiro (113402), Rodrigo Abreu (113402)

April 27, 2026

Contents

Chapter 1

Introduction and Scenario

1.1 Scenario Choice

This project follows **Scenario C - Omnichannel Commerce Core**. VerdeMart Retail started by using nopCommerce as its online storefront, but the business now needs more than a web shop. It wants web sales, warehouse execution, shipping, store operations, support, and customer visibility to work together as part of one commerce experience.

In this scenario, nopCommerce becomes the commerce core of that wider ecosystem. A web order should be reflected in operational systems such as warehouse and shipping, and changes that happen outside nopCommerce, such as stock or fulfillment updates, should become visible again in the customer and operator experience. This is important because those surrounding systems may be slow, stale, unavailable, or inconsistent, but the commerce experience still needs to remain useful and understandable.

1.2 Core Architectural Problem

Currently, nopCommerce mainly acts as the online shop: it owns the storefront, catalog browsing, cart, checkout, customer accounts, and order creation inside one web-centered platform. This works while the business is mostly operating through the online channel, but it does not fully address an omnichannel environment where warehouse execution, shipping, store operations, and support systems also influence the customer experience.

The core architectural problem is deciding how this online shop can evolve into a commerce core without assuming that every surrounding operational system is always fast, available, and consistent. The architecture must define what remains inside nopCommerce, what moves around it, and how the system behaves when warehouse or shipping state is delayed, stale, unavailable, or later recovered.

Chapter 2

Current-State Analysis

2.1 Overview of nopCommerce

2.2 Strengths

2.3 Limitations under Scenario

Chapter 3

Domain and Bounded Contexts

- 3.1 Domain Overview
- 3.2 Bounded Contexts
- 3.3 Responsibilities
- 3.4 Architecture Diagram

Chapter 4

Quality Attribute Scenarios

Quality attribute scenarios make the architectural drivers measurable. For the VerdeMart omnichannel scenario, the most important pressure is not only that nopCommerce must create orders correctly, but that the commerce experience must remain understandable while warehouse, shipping, store, and support systems are slow, stale, unavailable, or later recovering.

The scenarios below use the six-part structure from the course material: stimulus source, stimulus, artifact, environment, response, and response measure.

4.1 Architectural Drivers

The scenarios are driven by four architectural concerns:

- **Availability:** web orders must not be lost when warehouse or shipping systems fail.
- **Consistency:** inventory and order state may be temporarily stale, but inconsistencies must be visible and recoverable.
- **Performance:** customer-facing operations must stay responsive during backend synchronization and sales peaks.
- **Modifiability:** new operational channels must be integrated without rewriting checkout or sharing the nopCommerce database.

4.2 Scenario 1 - Availability under Warehouse Failure

Driver: Availability and operational continuity.

Part	VerdeMart scenario
Stimulus source	Warehouse execution system
Stimulus	The warehouse API takes more than 10 seconds to respond or rejects fulfillment requests for newly paid web orders.
Artifact	Fulfillment synchronization and the order status view shown to customers and operators
Environment	Friday afternoon peak traffic, with 3 times normal order volume, while the warehouse dependency is degraded but the web shop is still accepting orders
Response	The order is accepted and persisted locally; the fulfillment request is stored for asynchronous retry; the order is shown as awaiting warehouse confirmation; subsequent orders are not blocked by the degraded dependency.
Response measure	No paid order is lost; no duplicate fulfillment request is sent; pending fulfillment state is visible to support within 30 seconds; retry or escalation starts within 5 minutes.

4.3 Scenario 2 - Consistency under Stale Inventory

Driver: Consistency and recoverable inventory state.

Part	VerdeMart scenario
Stimulus source	Store point-of-sale system and warehouse stock service
Stimulus	A store sale reduces stock by 5 units, but the update reaches nopCommerce 3 minutes after the web catalog still showed those units as available.
Artifact	Inventory availability view used by product pages, cart validation, and checkout
Environment	Concurrent web and physical-store sales for the same product during a promotion, with external stock updates delayed by up to 3 minutes
Response	The commerce core marks stock data as stale, revalidates stock before checkout completion, preserves already-created orders for reconciliation, and records an inventory discrepancy for recovery.
Response measure	Product pages show stale stock status within 30 seconds of detection; overselling stays under 1% of affected order lines; inventory discrepancies are queued for reconciliation within 1 minute.

4.4 Scenario 3 - Performance during Omnichannel Order Bursts

Driver: Performance under customer-facing load.

Part	VerdeMart scenario
Stimulus source	Web customers, store operators, and customer support agents
Stimulus	A campaign creates 5 times normal traffic for product browsing, cart validation, order creation, and order-status queries.
Artifact	Storefront, checkout path, order status view, and integration event publishing path
Environment	Peak campaign period, with 5 times normal traffic and background synchronization still running
Response	The system prioritizes checkout writes, serves order-status reads from a projection, and processes warehouse and shipping synchronization asynchronously without blocking the customer request path.
Response measure	Checkout p95 latency remains below 2 seconds; order-status read p95 remains below 1 second; at least 99% of integration events are published or queued within 60 seconds.

4.5 Scenario 4 - Recoverability after Shipping Outage

Driver: Recoverability after external dependency failure.

Part	VerdeMart scenario
Stimulus source	Shipping provider or shipping adapter
Stimulus	Shipment label creation fails for 50 ready-to-dispatch orders and later becomes available again.
Artifact	Shipping integration workflow, retry queue, and order shipment status
Environment	Warehouse has packed orders, but the external shipping provider is unavailable for 20 minutes
Response	The system stores failed label requests, keeps orders in a clear awaiting-shipping state, avoids repeated synchronous calls from operators, and resumes processing automatically when the provider recovers.
Response measure	100% of failed label requests are retained; operators see the outage state within 30 seconds; 95% of queued labels are created within 10 minutes after provider recovery.

4.6 Scenario 5 - Modifiability for New Operational Channels

Driver: Modifiability and controlled integration growth.

Part	VerdeMart scenario
Stimulus source	VerdeMart business team and integration developers
Stimulus	A new store-operations system must receive order-ready-for-pickup events and send pickup-completed updates back to the commerce core.
Artifact	Integration layer, domain events, and order status update boundary
Environment	Existing web, warehouse, and shipping integrations are already in production
Response	The architecture allows the new channel to be added through a new adapter and event subscription without changing checkout logic or sharing the nopCommerce database with the external system.
Response measure	The new integration requires changes in no more than one adapter and one application-service boundary; no direct external database access is introduced; regression tests for checkout and existing integrations remain green.

4.7 Traceability to Architectural Drivers

Scenario	Architectural driver
Availability under warehouse failure	Availability, recoverability
Consistency under stale inventory	Consistency
Performance during omnichannel order bursts	Performance
Recoverability after shipping outage	Recoverability, availability
Modifiability for new operational channels	Modifiability

Chapter 5

Architectural Approach

5.1 Chosen Framework

The selected architectural design framework is **Attribute-Driven Design (ADD)**.

ADD fits this project because the VerdeMart scenario is mainly driven by measurable quality attribute pressures. The main question is not only which components should exist, but how the commerce core should behave when surrounding operational systems are delayed, stale, unavailable, or recovering.

In this project, ADD is applied as follows:

1. Identify the business and architectural drivers for the omnichannel commerce core.
2. Define quality attribute scenarios for the most important runtime pressures.
3. Select architectural tactics that satisfy those scenarios.
4. Decompose the target architecture into responsibilities, integration boundaries, and external adapters.
5. Validate the design through a focused runtime demonstration of degradation and recovery.

5.2 Justification

The chosen scenario requires nopCommerce to evolve from a web storefront into the commerce core of a wider operational ecosystem. This creates architectural pressure in five main areas: availability, consistency, performance, recoverability, and modifiability.

ADD is appropriate because each of these pressures can be expressed as a concrete quality attribute scenario. Warehouse failure drives the need for asynchronous fulfillment and retry. Stale inventory drives the need for inventory visibility, revalidation, and reconciliation. Order bursts drive the need to protect customer-facing paths from slow backend synchronization. Shipping outage drives the need for durable retry and explicit recovery state. New operational channels drive the need for adapters and stable integration boundaries.

This gives a direct trace from the problem to the architecture:

ADD input	Architectural consequence
Availability scenario	Avoid synchronous dependency on warehouse systems during checkout.
Consistency scenario	Introduce explicit inventory state, stale-state visibility, and reconciliation.
Performance scenario	Keep checkout and order reads separate from slow integration processing.
Recoverability scenario	Store failed integration work durably and retry after dependency recovery.
Modifiability scenario	Add external systems through adapters instead of direct database coupling.

5.3 Why Not ACDM or ADM as the Main Framework

ACDM was considered because it is useful for architectural decision-making, risk discussion, and stakeholder validation. However, it is not the main framework for this project because the strongest design input is the set of measurable quality attribute scenarios. The project must show runtime behavior under failure, delay, stale state, and recovery; ADD gives a more direct path from those scenarios to concrete architectural tactics and component responsibilities.

ACDM-style reasoning is still useful later in the project, especially when documenting ADRs and explaining rejected alternatives. For example, the decision to use asynchronous integration can be recorded as an ADR with alternatives such as direct synchronous calls or shared database access. However, the decision itself is primarily driven by the availability and recoverability scenarios, which makes ADD the better primary method.

ADM/TOGAF was also considered, but it is broader than necessary for this assignment. ADM is better suited to enterprise-wide transformation across business, data, application, and technology architecture. VerdeMart does have an omnichannel business context, but the scope of this work is a focused software architecture evolution of nopCommerce, not a full enterprise architecture programme.

5.4 How ADD Shapes the Target Architecture

Using ADD means that each architectural element in the target design should answer a specific quality attribute pressure. The surrounding systems are not added only because they exist in the business scenario; they are added because they create design pressure that nopCommerce must handle explicitly.

For this reason, the target architecture will emphasize asynchronous integration between nopCommerce and operational systems. It will also make status explicit for pending, degraded, failed, and recovered work, so that customers, support agents, and operators can understand what is happening during dependency failures. Warehouse, inventory, and shipping flows will use retry and reconciliation mechanisms instead of assuming that every external system is always available. External systems will be connected through adapters to avoid coupling them directly to nopCommerce internals, and data ownership will remain explicit so that surrounding systems do not share the nopCommerce database.

The next chapter uses these ADD-driven conclusions to define the target architecture.

Chapter 6

Target Architecture

6.1 Overview

6.2 Components

6.3 Interactions

6.4 Data Ownership

6.5 Main Architecture Diagram

Chapter 7

Architectural Decisions

7.1 ADR 1 - XXX

7.1.1 Context

7.1.2 Decision

7.1.3 Alternatives Considered

7.1.4 Consequences

7.2 ADR 2 - XXX

7.2.1 Context

7.2.2 Decision

7.2.3 Alternatives Considered

7.2.4 Consequences

7.3 ADR 3 - XXX

7.3.1 Context

7.3.2 Decision

7.3.3 Alternatives Considered

7.3.4 Consequences

Chapter 8

Risk and Validation Plan

Chapter 9

Evolution Roadmap

Chapter 10

Feasibility Spike

10.1 What Was Tested

10.2 What Worked

10.3 Evidence